Embedded AI

Author: [kevin Hector] **Date created:** 20/09/2022
Description: CNN based on MNIST

Setup

```
import numpy as np
import tensorflow
from tensorflow import keras
from tensorflow.keras import layers, models
```

Define the model

```
def build_model(input_shape):

        # Small CNN for MNIST recognition
        model = models.Sequential()

        # Dense layer
        model.add(layers.Conv2D(2, (3, 3), padding='same', activation='relu', input_shape=input_shape))
        model.add(layers.MaxPooling2D((2, 2), padding='valid'))
        model.add(layers.Flatten())

        # Dense layer
        model.add(layers.Dense(16, activation='relu'))

        # Output layer
        model.add(layers.Dense(10, activation='softmax'))

        return model
```

Prepare the data

```
# Model / data parameters
num_classes = 10
input_shape = (28, 28, 1)

# Load the data and split it between train and test sets
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# Scale images to the [0, 1] range
x_train = x_train.astype("float32") / 255
x_test = x_test.astype("float32") / 255
# Make sure images have shape (28, 28, 1)
x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)

x_val = x_train[50000:]
y_val = y_train[50000:]

x_train = x_train[:50000]
y_train = y_train[:50000]



print("x_train shape:", x_train.shape)
print(x_train.shape[0], "train samples")
print(x_test.shape[0], "test samples")
print(x_val.shape[0], "validation samples")


# convert class vectors to binary class matrices
y_train = keras.utils.to_categorical(y_train, num_classes)
y_test = keras.utils.to_categorical(y_test, num_classes)
y_val = keras.utils.to_categorical(y_val, num_classes)
```

```
x_train shape: (50000, 28, 28, 1)
50000 train samples
```

```
10000 test samples
10000 validation samples
```

Build the model

```
model = build_model(input_shape)
model.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 28, 28, 2)         20

 max_pooling2d (MaxPooling2D  (None, 14, 14, 2)        0
 )

 flatten (Flatten)           (None, 392)               0

 dense (Dense)               (None, 16)                6288

 dense_1 (Dense)             (None, 10)                170

=================================================================
Total params: 6,478
Trainable params: 6,478
Non-trainable params: 0
_________________________________________________________________
```

Train the model

```
batch_size = 128
epochs = 5

l_rate = 0.01
optimizer = tensorflow.keras.optimizers.Adam(lr=l_rate)

model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])


history = model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs, validation_data=(x_val, y_val), shuffle=True)
```

```
Epoch 1/5
/usr/local/lib/python3.7/dist-packages/keras/optimizer_v2/adam.py:105: UserWarning: The `lr` argument is deprecated, use `le
  super(Adam, self).__init__(name, **kwargs)
391/391 [==============================] - 13s 32ms/step - loss: 0.7831 - accuracy: 0.7764 - val_loss: 0.2658 - val_accuracy
Epoch 2/5
391/391 [==============================] - 13s 34ms/step - loss: 0.2570 - accuracy: 0.9239 - val_loss: 0.2003 - val_accuracy
Epoch 3/5
391/391 [==============================] - 13s 33ms/step - loss: 0.2053 - accuracy: 0.9381 - val_loss: 0.1718 - val_accuracy
Epoch 4/5
391/391 [==============================] - 12s 32ms/step - loss: 0.1785 - accuracy: 0.9467 - val_loss: 0.1602 - val_accuracy
Epoch 5/5
391/391 [==============================] - 12s 30ms/step - loss: 0.1610 - accuracy: 0.9515 - val_loss: 0.1459 - val_accuracy
```

Evaluate the trained model

```
score = model.evaluate(x_test, y_test, verbose=0)
print("Test loss:", score[0])
print("Test accuracy:", score[1])
```

```
Test loss: 0.1436065137386322
Test accuracy: 0.9577000141143799
```

Convert to h5 format

```
import numpy as np

np.save("MNIST_xtest_NN_C2_16_10.npy", x_test)
np.save("MNIST_ytest_NN_C2_16_10.npy", y_test)
model.save("MNIST_NN_C2_16_10.h5")
```